

MAXIMUS.**GUIDE D'INSTALLATION**

Deux modes de déploiement, une même exigence d'isolation.

Procédure complète pour l'instance centrale multi-entreprises
et l'instance dédiée/on-premise limitée à une seule
entreprise.

Document opérationnel — Laravel + PostgreSQL + React/Vite

Version correspondant au dépôt vérifié le 16 septembre 2026

Les secrets sont toujours fournis par l'opérateur ; aucun secret réel n'est inclus dans ce document.

Sommaire

1. Choisir le bon mode
2. Architecture et principes de sécurité
3. Prérequis communs
4. Installation A — instance centrale sur Render
5. Installation B — instance dédiée ou on-premise
6. Variables d'environnement
7. Exploitation quotidienne
8. Mises à jour, sauvegardes et restauration
9. Validation de sécurité et recette
10. Dépannage
11. Checklist de mise en service

Lecture rapide : le mode central est l'instance SaaS partagée qui conserve l'administration MAXIMUS. Le mode dédié/on-premise est une installation séparée qui n'accepte qu'un compte `company_admin` de l'entreprise configurée et refuse l'administration globale.

1. Choisir le bon mode

Besoin	Mode	Choix recommandé
MAXIMUS héberge plusieurs entreprises et l'équipe MAXIMUS administre le catalogue, les demandes et les paramètres plateforme.	<code>central</code>	Render avec PostgreSQL géré, disque persistant et domaine public.
Une entreprise veut une URL et une base séparées, sans administration globale MAXIMUS.	<code>dedicated</code>	VPS ou Render configuré avec une entreprise stable.
Installation dans le réseau interne ou sur un serveur contrôlé par le client.	<code>on_promise</code>	Même procédure que <code>dedicated</code> , avec accès réseau et certificats gérés localement.

Ne pas confondre abonnement et installation : l'abonnement détermine la facturation et les modules autorisés dans l'instance centrale. Une installation dédiée détermine une frontière serveur et doit être configurée par environnement ; le navigateur ne peut pas choisir son entreprise.

Règle de décision

- Utiliser `central` si plusieurs entreprises doivent cohabiter dans la même base.

- Utiliser `dedicated` si l'instance doit être réservée à une entreprise mais reste hébergée sur un VPS ou un cloud.
- Utiliser `on_premise` si le serveur est installé dans l'infrastructure de l'entreprise.
- Ne jamais mettre un même identifiant de base ou un même fichier `.env` en partage entre deux instances isolées.

2. Architecture et principes de sécurité

Le runtime actif est Laravel + Eloquent + PostgreSQL. Le frontend React/Vite est construit puis servi par le dossier public Laravel. Express/Drizzle restent présents pour la migration progressive et la comparaison de parité ; ils ne doivent pas devenir le serveur de production par accident.

Central

- `MAXIMUS_DEPLOYMENT_MODE=central`
- Connexion possible pour `maximus_admin`, `company_admin` et employés autorisés.
- Routes plateforme accessibles selon permissions.
- Inscription publique et onboarding disponibles selon le réglage global.

Isolé

- `MAXIMUS_DEPLOYMENT_MODE=dedicated` ou `on_premise`.
- Entreprise fixée par `MAXIMUS_INSTALLATION_COMPANY_ID`.
- Un seul compte initial `company_admin`.
- Routes centrales et données publiques d'autres entreprises refusées.

Contrôles serveur appliqués

1. **Authentification** : une connexion isolée ne recherche que les utilisateurs de l'entreprise configurée.
2. **Session** : une session d'un autre tenant est refusée par `AuthenticateMaximus`.
3. **Routes centrales** : elles renvoient `404 CENTRAL_INSTALLATION_ONLY` en mode isolé.
4. **Vitrines publiques** : boutiques, domaines, images, profils publics et Transport sont filtrés avant le contrôleur.
5. **Bootstrap** : `maximus:install-company` est idempotent et ne crée jamais `maximus-admin`.
6. **Navigation** : l'interface masque l'inscription et l'accès central, mais la sécurité ne dépend pas du frontend.

3. Prérequis communs

Composant	Central Render	VPS/local isolé
Code	Dépôt GitHub relié à Render.	Workspace cloné ou archive complète du monorepo.
PHP	Fourni par l'image <code>php:8.2-apache</code> .	PHP 8.2+, extensions PDO PostgreSQL, mbstring, XML, curl et zip.
Node	Fourni par l'étape Docker Node 24.	Node.js 24, Corepack et pnpm.
Base	PostgreSQL Render lié par <code>DATABASE_URL</code> .	PostgreSQL accessible, base et utilisateur créés.
Fichiers	Disque Render monté sur <code>/var/data</code> .	Répertoire persistant pour <code>DIGITAL_STORAGE_PATH</code> et sauvegardes.
HTTPS	Domaine Render ou domaine personnalisé.	Reverse proxy HTTPS recommandé ; HTTP seulement pour un test local.

Avant toute production : ne pas copier un fichier `.env` de démonstration, ne pas publier les mots de passe dans Git, ne pas utiliser le disque éphémère pour les fichiers numériques et ne pas réutiliser une base entre central et dédié.

4. Installation A — instance centrale sur Render

4.1 Créer le service

1. Relier le dépôt GitHub contenant le monorepo à Render.
2. Utiliser le fichier `render.yaml` ou créer un Web Service Docker.
3. Vérifier que le Dockerfile utilisé est `Dockerfile.render` et que le contexte est la racine du dépôt.
4. Vérifier le chemin de santé : `/api/healthz`.
5. Conserver une base PostgreSQL Render distincte pour la production.

4.2 Variables obligatoires

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://votre-domaine
APP_KEY=<secret Laravel généré, jamais affiché dans Git>
MAXIMUS_DEPLOYMENT_MODE=central
DATABASE_URL=<connexion PostgreSQL Render>
DB_CONNECTION=pgsql
ADMIN_USER=<email administrateur MAXIMUS>
ADMIN_PASSWORD=<mot de passe d'au moins 8 caractères>
DIGITAL_STORAGE_PATH=/var/data/digital-products
SESSION_DRIVER=database
SESSION_SECURE_COOKIE=true
CACHE_STORE=database
LOG_CHANNEL=stderr
```

4.3 Variables paiement

Les variables DiamanoPay sont optionnelles au démarrage mais nécessaires pour encaisser ou retirer des fonds. Les renseigner uniquement dans les secrets Render :

```
DIAMANOPAY_BASE_URL=https://api.diamanopay.com
DIAMANOPAY_ACCESS_TOKEN=<secret>
DIAMANOPAY_CLIENT_ID=<secret>
DIAMANOPAY_CLIENT_SECRET=<secret>
DIAMANOPAY_WEBHOOK_SECRET=<secret>
```

4.4 Séquence de démarrage

1. Docker construit le frontend avec `PORT=10000` et `BASE_PATH=/` .
2. Composer installe Laravel sans les dépendances de développement.
3. Render monte le disque persistant et le script vérifie les permissions.
4. Le script attend PostgreSQL puis applique `php artisan migrate --force` .
5. Le mode central exécute `php artisan maximus:provision-admin` .
6. Apache sert `/app/laravel/public` sur le port 10000.
7. Render appelle `GET /api/healthz` .

4.5 Première validation

```
curl -fsS https://votre-domaine/api/healthz
curl -fsS https://votre-domaine/api/installation
```

La réponse d'installation centrale doit contenir `"mode":"central"` , `"companyOnly":false` , `"ready":true` et `"adminLoginEnabled":true` .

4.6 Domaine et cookies

- Déclarer le domaine personnalisé dans Render et attendre son certificat.
- Mettre exactement cette URL dans `APP_URL` .
- Conserver `SESSION_SECURE_COOKIE=true` en HTTPS.
- Tester une connexion, un rafraîchissement de page et une déconnexion.

5. Installation B — instance dédiée ou on-premise

5.1 Préparer le serveur

```
git clone <depot-prive> maximus
cd maximus
php -v
composer --version
node --version
corepack --version
psql --version
curl --version
```

Le script ne remplace pas le gestionnaire de paquets du système : installer PHP, les extensions, Composer, Node et PostgreSQL selon la distribution du serveur, puis relancer les vérifications ci-dessus.

5.2 Créer PostgreSQL

Exemple Debian/Ubuntu pour une base locale. Choisir un mot de passe unique et le conserver dans un gestionnaire de secrets :

```
sudo -u postgres createuser --pwprompt maximus
sudo -u postgres createdb --owner=maximus maximus
sudo -u postgres psql -d maximus -c \
    "ALTER ROLE maximus SET client_encoding TO 'UTF8';"
```

Pour une base distante, demander au DBA l'hôte, le port, le nom de base, l'utilisateur et les règles réseau ; ne pas ouvrir PostgreSQL à Internet sans filtrage.

5.3 Lancer l'installateur interactif

```
./scripts/install-maximus-instance.sh
```

Le script demande notamment :

- l'identifiant stable de l'entreprise ; le garder inchangé lors des relances ;
- le nom, le responsable, l'email, le pays et le secteur ;
- la connexion PostgreSQL ;
- l'email et le mot de passe initial du `company_admin` ;
- l'URL publique et les modules à activer.

5.4 Exécution automatisée

Éviter de passer des mots de passe dans les arguments de commande. Fournir les variables dans l'environnement du processus ou un mécanisme de secrets du système, avec un fichier protégé en `600` :

```
export MAXIMUS_DEPLOYMENT_MODE=dedicated
export MAXIMUS_INSTALLATION_COMPANY_ID=<identifiant-stable>
export MAXIMUS_COMPANY_NAME='Entreprise Exemple'
export MAXIMUS_COMPANY_MANAGER='Responsable Exemple'
export MAXIMUS_COMPANY_EMAIL='contact@example.test'
export MAXIMUS_ADMIN_USER='admin@example.test'
read -r -s MAXIMUS_ADMIN_PASSWORD
export MAXIMUS_ADMIN_PASSWORD
read -r -s DB_PASSWORD
export DB_PASSWORD
export DB_HOST=127.0.0.1 DB_PORT=5432
export DB_DATABASE=maximus DB_USERNAME=maximus
export APP_URL=https://erp.example.test
export MAXIMUS_INSTALLATION_MODULES=commerce,ecommerce,stocks,presences

./scripts/install-maximus-instance.sh --non-interactive
```

5.5 Mode on-premise

Utiliser la même commande en remplaçant le mode :

```
export MAXIMUS_DEPLOYMENT_MODE=on_premise
./scripts/install-maximus-instance.sh --non-interactive
```

Le mode on-premise reste serveur-isolé. Prévoir une URL interne stable, un certificat TLS de l'entreprise, des sauvegardes hors serveur et une règle de pare-feu autorisant uniquement les postes nécessaires.

5.6 Démarrer l'API

Pour une vérification locale uniquement :

```
cd artifacts/api-server/laravel
php -S 0.0.0.0:8080 server.php
```

Pour la production, utiliser un service supervisé et un reverse proxy HTTPS (Apache ou Nginx avec PHP-FPM). Le serveur PHP intégré est utile pour le test, mais ne doit pas être exposé directement à Internet comme stratégie de production durable.

5.7 Validation de l'instance isolée

```
curl -fsS https://erp.example.test/api/healthz
curl -fsS https://erp.example.test/api/installation
```

La réponse doit indiquer `"mode": "dedicated"` ou `"mode": "on_premise"`, `"companyOnly": true`, `"ready": true`, `"adminLoginEnabled": false` et l'entreprise configurée.

Une connexion avec l'email du compte `company_admin` doit réussir. Une connexion avec un ancien compte `maximus_admin` doit être refusée.

6. Variables d'environnement

Variable	Central	Dédié / on-premise	Rôle
MAXIMUS_DEPLOYMENT_MODE	central	dedicated ou on_premise	Choisit la frontière serveur.
MAXIMUS_INSTALLATION_COMPANY_ID	Vide	Obligatoire et stable	Tenant fixé côté serveur.
MAXIMUS_COMPANY_NAME	Non requis	Obligatoire à l'amorçage	Nom de l'entreprise créée/mise à jour.
MAXIMUS_COMPANY_EMAIL	Non requis	Obligatoire à l'amorçage	Email de contact de l'entreprise.
MAXIMUS_INSTALLATION_MODULES	Non utilisé	Liste séparée par virgules	Modules publiés autorisés au provisionnement.
ADMIN_USER/PASSWORD	Obligatoires	Fallback possible	Compte global central uniquement.
MAXIMUS_ADMIN_USER/PASSWORD	Non requis	Recommandées	Compte company_admin isolé.
DATABASE_URL	Render	Optionnel si DB_HOST est utilisé	URL PostgreSQL complète.
DB_HOST/PORT/DATABASE/USERNAME/PASSWORD	Alternative	Recommandées	Paramètres PostgreSQL séparés.
APP_KEY	Obligatoire	Générée par le script si absente	Chiffrement Laravel ; ne jamais renouveler sans plan de migration.
APP_URL	Obligatoire	Obligatoire	URL canonique et construction de liens/webhooks.
DIGITAL_STORAGE_PATH	/var/data/digital-products	Disque persistant	Fichiers numériques et ressources publiques.

Règle secrets : ne jamais mettre `APP_KEY`, mots de passe PostgreSQL, mots de passe admin ou identifiants DiamanoPay dans Git, un ticket, une capture ou une ligne de commande visible par d'autres utilisateurs.

7. Exploitation quotidienne

Contrôles de santé

```
curl -fsS https://votre-domaine/api/healthz
curl -fsS https://votre-domaine/api/installation
```

Surveiller au minimum le code HTTP, `ok`, la disponibilité PostgreSQL et le champ `mode`. En installation isolée, toute réponse affichant `central` est une anomalie de configuration.

Journaux

- Render : logs du Web Service et de l'instance PostgreSQL.
- VPS : logs du service systemd, du reverse proxy et `storage/logs/laravel.log` selon la configuration.
- Ne pas journaliser les corps de requêtes contenant des mots de passe, cookies ou tokens.

Accès administrateur

- Central : utiliser uniquement le compte global géré par `maximus:provision-admin` pour l'administration plateforme.
- Isolé : utiliser le compte `company_admin` créé par `maximus:install-company`.
- Après changement de mot de passe ou de périmètre, les sessions existantes sont invalidées.

8. Mises à jour, sauvegardes et restauration

Avant mise à jour

1. Confirmer le mode et l'entreprise avec `/api/installation`.
2. Faire un dump PostgreSQL.
3. Sauvegarder le répertoire de fichiers numériques.
4. Noter le commit déployé et vérifier que les migrations sont présentes.
5. Prévoir une fenêtre de maintenance pour les changements de schéma.

Sauvegarde PostgreSQL

```
pg_dump --format=custom --file=maximus-$(date +%F-%H%M).dump \
--host="$DB_HOST" --port="$DB_PORT" \
--username="$DB_USERNAME" "$DB_DATABASE"
```

Protéger le dump, le stocker hors du serveur et tester périodiquement une restauration sur une base séparée.

Mise à jour VPS

```
git fetch --all --tags
git checkout <version-validée>
cd artifacts/api-server/laravel
composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader
php artisan down --render="errors:503" || true
php artisan migrate --force --no-interaction
cd ../../..
PORT=10000 BASE_PATH=/ NODE_ENV=production \
  corepack pnpm --filter @workspace/maximus run build
cp -R artifacts/maximus/dist/public/. artifacts/api-server/laravel/public/
cd artifacts/api-server/laravel
php artisan up
php artisan maximus:install-company --no-interaction
```

Ne pas exécuter le bootstrap central sur une instance isolée. Après la migration, relancer les tests et la sonde de santé avant de rouvrir le trafic.

Restauration

1. Arrêter le trafic.
2. Restaurer le code précédent.
3. Restaurer la base uniquement si la migration a rendu le schéma incompatible et si le dump est validé.
4. Restaurer les fichiers numériques correspondant au même point de sauvegarde.
5. Vérifier mode, entreprise, santé, connexion et données avant réouverture.

9. Validation de sécurité et recette

Central

- ☐ `/api/installation` retourne `central` et autorise l'inscription selon le réglage.
- ☐ L'administrateur global peut se connecter.
- ☐ Une entreprise active peut se connecter selon ses droits.
- ☐ Les routes plateforme protégées sont accessibles seulement à l'acteur prévu.
- ☐ Les paiements et webhooks utilisent les secrets de production.
- ☐ Une restauration PostgreSQL a été testée.

Dédié / on-premise

- ☐ `/api/installation` retourne le mode isolé et le bon identifiant d'entreprise.
- ☐ Le compte `company_admin` se connecte.
- ☐ Un ancien `maximus_admin` est refusé.
- ☐ `/api/registration-catalog` et `/api/platform-settings/public-registration` retournent `404 CENTRAL_INSTALLATION_ONLY`.
- ☐ Une session d'un autre tenant retourne `403 INSTALLATION_COMPANY_ONLY`.

- ☐ `/api/shop/<slug-autre-entreprise>` , les images et Transport public retournent `404` `INSTALLATION_COMPANY_ONLY` .
- ☐ La boutique de l'entreprise dédiée fonctionne avec son slug ou son domaine autorisé.
- ☐ La base et les fichiers sont séparés de l'instance centrale.

Commandes de vérification du dépôt

```
cd artifacts/api-server/laravel
php artisan test --compact
cd ../../..
pnpm --filter @workspace/maximus run typecheck
pnpm --filter @workspace/maximus test
PORT=4173 BASE_PATH=/maximus pnpm --filter @workspace/maximus run build
sh -n render-entrypoint.sh
bash -n scripts/install-maximus-instance.sh
git diff --check
```

10. Dépannage

Symptôme	Cause probable	Action
Le service refuse de démarrer avec <code>APP_KEY</code> .	Secret absent ou mal configuré.	Définir <code>APP_KEY</code> , sans l'afficher dans les logs, puis redémarrer.
Migration PostgreSQL en boucle sur Render.	Base non prête, URL incorrecte ou règle réseau.	Vérifier le binding <code>DATABASE_URL</code> , l'état DB et les logs, puis laisser le mécanisme de retry terminer.
Installation dédiée : <code>INSTALLATION_NOT_CONFIGURED</code> .	Identifiant d'entreprise absent.	Définir <code>MAXIMUS_INSTALLATION_COMPANY_ID</code> , migrer puis relancer le bootstrap.
Installation dédiée : <code>COMPANY_ONLY</code> .	Session d'un autre tenant ou mauvaise base.	Supprimer le cookie, se reconnecter avec le compte entreprise et vérifier l'environnement de base.
Vitrine d'un autre slug en 404.	Comportement attendu en mode isolé.	Utiliser le slug de l'entreprise configurée ou enregistrer son domaine actif.
Frontend blanc après déploiement.	Frontend non construit ou mauvais <code>BASE_PATH</code> .	Rejouer le build avec <code>BASE_PATH=/</code> pour Laravel et vérifier <code>public/index.html</code> .
Fichiers numériques perdus après redémarrage.	Stockage éphémère.	Configurer un disque persistant et sauvegarder <code>DIGITAL_STORAGE_PATH</code> .
Cookie non conservé en HTTPS.	<code>SESSION_SECURE_COOKIE</code> incohérent ou proxy mal configuré.	Mettre la valeur à <code>true</code> , vérifier TLS et les en-têtes proxy.
DiamanoPay indisponible.	Secrets manquants, paiement entreprise désactivé ou webhook inaccessible.	Vérifier les secrets, le statut de paiement, l'URL webhook publique et les logs sans exposer les tokens.

11. Checklist de mise en service

- ☐ Le mode serveur est explicite et correspond à l'architecture choisie.
- ☐ `APP_KEY`, bases et mots de passe sont dans un gestionnaire de secrets.
- ☐ La base PostgreSQL est dédiée à l'environnement et sauvegardée.
- ☐ Les fichiers publics et numériques sont sur un stockage persistant.
- ☐ HTTPS, domaine, cookies sécurisés et URL canonique sont vérifiés.
- ☐ Les migrations passent sans erreur.
- ☐ `/api/healthz` et `/api/installation` renvoient les valeurs attendues.
- ☐ Le parcours de connexion correspondant au mode fonctionne.
- ☐ Les routes centrales sont refusées sur une instance isolée.
- ☐ Les boutiques, domaines, images et Transport des autres entreprises sont refusés en mode isolé.
- ☐ Un dump et une restauration de test ont été réalisés.
- ☐ Les logs ne contiennent aucun secret.
- ☐ La procédure de mise à jour et de retour arrière est écrite et testée.

État attendu en fin de procédure : l'instance centrale reste multi-entreprises et administrable par MAXIMUS ; l'instance dédiée/on-premise ne voit et ne sert que l'entreprise configurée, y compris pour ses parcours publics.

MAXIMUS — Guide d'installation central et dédié/on-premise. Les valeurs entre chevrons sont des exemples à remplacer. Ne jamais copier de secret réel dans ce document.